

ASL Sign Recognition — Dev Log

A running log of the build, the problems hit along the way, and how they got fixed. Started as a software proof-of-concept for a physical ASL interpretation device, built for my deaf mom.

The idea

Webcam → hand/pose/face landmarks (MediaPipe) → sequence model (GRU) → predicted word, printed live. Starting vocabulary: 10 words — dad, happy, hello, later, mom, no, please, sad, thankyou, yes.

Setting up the environment (more painful than expected)

On an Apple Silicon Mac, `mediapipe`, `tensorflow`, and `opencv` turned out to have a narrow window of versions that actually work together:

- `mediapipe >= 1.0` dropped the legacy `solutions.holistic` API entirely (Tasks API only now), so had to pin `mediapipe==0.10.14`.
- `tensorflow 2.20` wants `protobuf >= 5.28`, which conflicts with `mediapipe's protobuf < 5` requirement.
- `opencv-python 5.0` wants `numpy >= 2`, which conflicts with `tensorflow 2.15's numpy < 2` pin.
- Even with the right `tensorflow==2.15.0` pin, a plain `pip install` left `tensorflow/__init__.py` missing on arm64 — had to force-reinstall `tensorflow-macos==2.15.0` alongside it.

Working combination pinned in `requirements.txt`: `mediapipe==0.10.14`, `tensorflow==2.15.0`, `opencv-python==4.10.0.84`, `protobuf==4.25.9`.

Getting training data (Kaggle)

Used Google's "Isolated Sign Language Recognition" Kaggle dataset (landmark-only, no raw video — collected via the PopSign app, from parents of deaf children learning ASL at home, not necessarily fluent/certified signers — this detail mattered a lot later).

Kaggle auth had its own trap: the new-style `KGAT_...` API tokens generated from the main "API Tokens" page aren't supported by the `kaggle` or `kagglehub` pip packages — both only read the classic `kaggle.json` format, generated from a separate "Legacy API Key" section. Also needed

phone verification and to formally join the competition before downloads worked (silent 403s otherwise).

Then hit Kaggle's per-account rate limit downloading with 12 concurrent workers — it was still in effect hours later even at low concurrency. Rather than wait it out, training proceeded on the 446 files (of 3807 available for these 10 words) that had already downloaded.

Feature engineering

MediaPipe Holistic emits 543 landmarks/frame (468 face + 33 pose + 21+21 hands). For a ~450-sample dataset, that's way more parameters than the data could support, so the feature set was deliberately reduced:

- Dropped z entirely (x, y only)
- Kept both hands in full (21 pts each) — the main signal for handshape
- Only 6 pose points (shoulders/elbows/wrists, not all 33)
- Only 13 face points (nose/mouth/cheek area, not all 468)
- Final feature vector: 122-dim (61 landmarks × 2 coords)

Sequences padded/subsampled to a fixed 48 frames (~p90 of the observed sequence lengths; median was 18, p95 was 53).

First training run

GRU(64) → GRU(32) → Dense classifier. 446 training sequences, 55% validation accuracy (vs. 10% random chance for 10 classes). Live testing showed it was strongest on stationary/distinct handshapes (dad, mom, hello, later, please) and weak on motion-heavy signs (no, yes, sad, thankyou) — the obvious read was "not enough data," but that turned out to be only half the story.

The real bug: some Kaggle labels don't match modern ASL

Built `src/visualize_samples.py` — renders a training clip's landmark sequence as an animated 2D skeleton GIF (hand connections + arm/shoulder points), so a human who actually knows ASL can sanity-check whether a label matches the sign.

That's how this got caught: the Kaggle "later" clips were using an outdated sign variant, not the modern one (right hand forms an L held upward, then rotates 90° down, pointer toward camera). And "no" — despite having a totally normal sample count (~36-44 clips, same ballpark as working classes) — never got predicted *at all* across multiple live test sessions. That ruled out "not enough

data" as the explanation for "no" specifically; the label itself was the problem, most likely because the dataset's non-fluent-signer participants didn't consistently produce a clean, correct version of that sign.

The fix: record your own replacement clips

Built `src/record_samples.py` — opens the webcam, same MediaPipe setup as live inference, lets you record N reps of yourself signing a word correctly (SPACE to record after a countdown), and saves them to `data/recorded/<word>/`.

Extended `src/prepare_data.py` so that any word with files under `data/recorded/<word>/` has **all** of its Kaggle rows dropped and replaced entirely by the self-recorded clips — not merged in, since the goal was fixing a wrong label, not diluting bad data with a little good data.

Lessons learned re-doing "later," "no," and "yes"

- **7 recorded clips wasn't enough** — even with the correct sign now, "later" still never got predicted live until bumped up to 36 clips (then hit 46-99% confidence consistently). Rule of thumb: match the sample count of the strongest existing classes (~35-45), not just "some."
- **Relative class size matters, not just absolute** — after fixing "yes" with 28-29 clips, it kept losing to "later" (36 clips) in live testing, even though "yes" itself was no longer mislabeled. "Later" being more confidently learned let it dominate ambiguous predictions. Bumping "yes" to 41 clips (more than "later") fixed it.
- **One contaminated clip matters** — deleting a single "yes" recording that accidentally had my other arm in frame gave a measurable accuracy bump on its own.
- Final result: validation accuracy went from 55% → ~67%, despite the total training set actually *shrinking* (446 Kaggle clips → ~187 Kaggle + ~106 self-recorded) once three classes were fully swapped over. Fixing bad labels beat adding more of the same bad data.

Live inference UX: the "stickiness" bug

Original design: buffer 48 frames, predict once, clear the buffer, wait for a completely fresh 48 frames before predicting again. Worked, but felt slow (~1.5-2s dead air between predictions).

First attempt at speeding it up: never clear the buffer, just slide it forward and predict every 8 new frames. Cycling felt faster, but introduced a much worse bug — after finishing a sign, the old frames lingered in the window for several more prediction cycles and kept "overruling" whatever was signed next (e.g. "no" would keep getting predicted for a few cycles after switching to "yes" or

"dad"). This wasn't just visual sign similarity — it was literally leftover frames from the completed gesture still dominating the window.

Fix: buffer is a plain list, not a sliding deque. Once at least 24 frames are collected, it attempts a prediction every 4 new frames (zero-padded up to the model's expected 48-frame input, same as how short Kaggle clips get padded at training time). Critically, **the buffer is cleared after every prediction attempt, confident or not** — so a finished sign's frames never leak into the next one. This keeps the faster cadence without the cross-contamination.

It also didn't work for my mom

The whole point of this project is for my mom, so I had her try it. The classes I'd just fixed by re-recording myself (later/no/yes) worked much worse for her than for me, even when she signed them correctly. A small model trained on one person's exact body proportions, camera distance, and signing style overfits to that person — it doesn't automatically generalize to someone else. The Kaggle-sourced classes (which had many different people in the training data) transferred better, which in hindsight was a clue.

Fixed two ways at once:

1. **Data augmentation** — for every self-recorded clip, generate a couple of extra copies with a small random rotate/scale/shift jitter (same transform applied across the whole clip so the motion stays coherent). Simulates body-size and camera-position variation without needing more real footage.
2. **Actually recording my mom** — had her record 4-5 real reps of each fixed word herself, added alongside mine (not replacing — both signers' clips coexist for a class).

Together, this took those words from "works for me" to "works for both of us." Val accuracy went from 67% → 79%. Neither change alone would likely have been enough — augmentation only adds geometric variation, not real signing-style diversity.

Expanding the vocabulary: "minemy" and the pronoun problem

Wanted to start building toward actual sentences, not just single words. ASL often collapses what are separate words in English into one sign — "mine" and "my" are the same physical sign (open palm flat on the chest), so the Kaggle dataset (250 words total, not just the 10 I'd been using) has it as a single combined class: `minemy`. Same idea for `hesheit` (he/she/it) and `weus` (we/us).

No local Kaggle data existed for `minemy` at all (it wasn't part of the original download), and after last time's rate-limiting pain, I skipped Kaggle entirely and just recorded it from scratch — 32 clips, no

Kaggle data whatsoever. It worked fine, which was a useful discovery on its own: self- recording doesn't have to be a patch for bad Kaggle data, it can be the primary source from the start.

Whack-a-mole: fixing one class breaks another

Adding `minemy` immediately started stealing predictions that should have been "no" — a live test showed five real "no" attempts in a row misread as `minemy / later / yes` before a real "no" finally landed. Turned out `minemy` only had 25 real clips vs. 34-44 for the other re-recorded classes — a smaller class can end up with an oddly-shaped decision boundary that fires confidently in the wrong places. Bumping it to 32 fixed that particular confusion.

Then, immediately after, **"dad" and "mom" stopped being predicted at all** — two words that hadn't been touched since the very first training run. The cause wasn't their data — it was the augmentation from the "fix it for mom" work above. Self-recorded classes now had 2 extra augmented copies per clip, giving them ~90-130 training examples each vs. ~38-49 for untouched Kaggle classes. The model was just biased toward whichever classes it had seen more of, in raw volume.

Fixed with **inverse-frequency class weighting** during training — each class's contribution to the loss gets scaled by how rare it is (`class_weight` in Keras' `model.fit`), directly counteracting the volume imbalance without touching any data. "Dad" came back immediately after retraining with weighting on.

Turned out "mom" itself also had a real Kaggle-label problem underneath (same "never gets predicted despite normal sample count" signature as the original "no" bug) — recording just 5 of my own clips fixed it completely (0% → 85-100%). "Dad" and "happy" needed the same treatment and also only took 5 clips each — a fraction of what "later"/"no"/"yes" needed (30-40). Not entirely sure yet why some words need so few reps and others need so many; something to watch for on future words.

An open bug: idle hands read as a confident wrong answer

Repeatedly noticed long runs of an identical prediction/confidence (twenty straight "no (45%)" in one session, later "later (51%)" in another) — these line up with stretches where hands were just resting between actual signs, not real attempts. The classifier has no "nothing is happening" option, so a static idle pose gets mapped to whatever class the decision boundary considers closest, often with deceptively high confidence. Not fixed yet — the plan is a motion-detection guard that skips prediction entirely when the buffered frames show near-zero hand movement, rather than always classifying the window.

Current state

11-word vocabulary (dad, happy, hello, later, minemy, mom, no, please, sad, thankyou, yes), ~89% validation accuracy (up from the original 55%). Working toward simple sentences — "MOM, MINEMY HAPPY" is achievable with the current vocab. Git checkpoint committed at each major milestone so this state is never at risk of being lost.

Possible next steps

- Fix the idle-pose false-confidence bug (motion-detection guard before prediction)
- Check "thankyou" the same way — flagged as weak in the most recent testing
- Add more sentence-building words from the 250-word Kaggle vocab (hungry, thirsty, like, go, hesheit, weus, yourself are good candidates)
- Backfill more of the original 3807 available Kaggle files (only 446 were downloaded before hitting the rate limit) for the classes still using Kaggle data
- Explore a browser-based version (TensorFlow.js + MediaPipe Tasks Vision JS) so it's accessible as a website instead of a local Python script — the model itself is small enough that this would run fully client-side, no server needed. Main work would be re-implementing the landmark-extraction feature layout against the browser MediaPipe API, which is structured differently from the Python `solutions.holistic` API used here.